Fundamentals of Machine Learning

A sample knowledge-base document for building and testing a basic RAG (Retrieval-Augmented Generation) pipeline.

1. What Is Machine Learning?

Machine learning (ML) is a branch of artificial intelligence that enables systems to learn patterns from data and improve their performance on a task without being explicitly programmed for every rule. Instead of hand-coding logic, an ML system is given examples (data) and an objective (a loss function), and it adjusts its internal parameters to minimize error on that objective. The three broad categories of machine learning are supervised learning, unsupervised learning, and reinforcement learning. Each differs in the type of feedback the learning algorithm receives.

2. Supervised Learning

In supervised learning, the model is trained on labeled data: each input example is paired with a correct output. The goal is to learn a mapping from inputs to outputs that generalizes well to unseen data. Common supervised learning tasks include classification (predicting a discrete category, such as spam versus not-spam) and regression (predicting a continuous value, such as house price). Popular supervised learning algorithms include linear regression, logistic regression, decision trees, random forests, support vector machines, and neural networks. Model quality is typically measured using metrics such as accuracy, precision, recall, F1-score for classification, and mean squared error or R-squared for regression.

3. Unsupervised Learning

Unsupervised learning works with unlabeled data. The algorithm tries to find structure, patterns, or groupings in the data without being told the correct answer in advance. Common unsupervised tasks include clustering (grouping similar data points together, e.g., K-means or hierarchical clustering), dimensionality reduction (compressing data into fewer dimensions while preserving structure, e.g., PCA or t-SNE), and anomaly detection (identifying data points that deviate significantly from the norm). Unsupervised learning is often used for exploratory data analysis, customer segmentation, and as a preprocessing step before supervised learning.

4. Reinforcement Learning

Reinforcement learning (RL) involves an agent that learns to make decisions by interacting with an environment. The agent takes actions, receives rewards or penalties, and updates its policy to maximize cumulative reward over time. Key concepts in RL include the state, action, reward, policy, and value function. Well-known RL algorithms include Q-learning, policy gradients, and actor-critic methods. RL has been used successfully in game playing (e.g., AlphaGo), robotics, and resource

management.

5. Neural Networks and Deep Learning

A neural network is composed of layers of interconnected nodes, or neurons, each applying a weighted sum of its inputs followed by a nonlinear activation function. Deep learning refers to neural networks with many layers, which allows them to learn hierarchical representations of data. Convolutional neural networks (CNNs) are well-suited for image data because they exploit spatial locality through convolutional filters. Recurrent neural networks (RNNs) and their variants, such as LSTMs and GRUs, are designed for sequential data like text or time series. Transformers, introduced in 2017, replaced recurrence with a self-attention mechanism and now form the backbone of most state-of-the-art language models.

6. Training, Validation, and Test Sets

To evaluate a machine learning model fairly, the available data is usually split into three parts: a training set used to fit the model's parameters, a validation set used to tune hyperparameters and monitor for overfitting during development, and a test set used only once at the end to estimate real-world performance. A common split ratio is 70 percent training, 15 percent validation, and 15 percent test, though this varies with dataset size. Cross-validation, such as k-fold cross-validation, is often used when data is limited, rotating which portion of the data serves as the validation set.

7. Overfitting and Regularization

Overfitting occurs when a model learns the noise and specific quirks of the training data rather than the underlying general pattern, resulting in poor performance on new data. Underfitting is the opposite problem, where the model is too simple to capture the pattern at all. Regularization techniques help control overfitting. L1 regularization (Lasso) adds a penalty proportional to the absolute value of the weights, encouraging sparsity. L2 regularization (Ridge) adds a penalty proportional to the square of the weights, discouraging large weights. Dropout randomly disables neurons during training in neural networks to prevent co-adaptation. Early stopping halts training once validation performance stops improving.

8. What Is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation is a technique that combines a retrieval system with a generative language model. Instead of relying solely on knowledge baked into the model's parameters during training, a RAG system first retrieves relevant documents or passages from an external knowledge base at query time, then feeds those retrieved passages into the language model as additional context. This allows the model to answer questions using up-to-date or domain-specific information it was never explicitly trained on, and it reduces hallucination by grounding responses in retrieved evidence.

9. Building a Basic RAG Pipeline

A basic RAG pipeline typically has four stages. First, ingestion: documents are collected and split into smaller chunks, since language models have limited context windows and smaller chunks improve retrieval precision. Second, embedding: each chunk is converted into a numeric vector using an embedding model, capturing its semantic meaning. Third, storage and retrieval: the vectors are stored in a vector database or index (such as FAISS, Chroma, or Pinecone), and at query time the user's question is also embedded and compared against stored vectors using a similarity metric like cosine similarity to find the most relevant chunks. Fourth, generation: the retrieved chunks are inserted into a prompt along with the original question, and a language model generates the final answer grounded in that context.

10. Chunking Strategies for RAG

How a document is split into chunks strongly affects retrieval quality. Fixed-size chunking splits text into equal-length segments, often with some overlap between consecutive chunks to preserve context across boundaries. Sentence or paragraph-based chunking splits at natural language boundaries, which tends to keep semantically coherent units together. Recursive character splitting tries larger units first (like paragraphs) and falls back to smaller units (like sentences or words) if a chunk is still too large. A common starting point is chunks of 200 to 500 tokens with an overlap of 10 to 20 percent, though the ideal size depends on the embedding model and the nature of the source documents.

11. Evaluating a RAG System

RAG systems are typically evaluated on both retrieval quality and generation quality. Retrieval metrics include precision at k and recall at k, which measure whether the relevant chunks appear in the top k retrieved results. Generation quality can be evaluated using faithfulness (does the answer stay grounded in the retrieved context), relevance (does the answer address the question), and, when reference answers exist, metrics like ROUGE or BLEU. Human evaluation and LLM-based evaluation (using a separate model to judge answer quality) are also widely used in practice.

12. Common Pitfalls

Several issues commonly reduce RAG system quality. Poor chunking can split relevant information across chunk boundaries, causing retrieval to miss it. Using an embedding model that is a poor fit for the domain (for example, a general-purpose embedding model on highly technical legal text) can reduce retrieval accuracy. Retrieving too few chunks can starve the model of needed context, while retrieving too many can dilute relevant information and exceed context window limits. Finally, without proper grounding instructions in the prompt, the language model may still hallucinate details not present in the retrieved context.